

robocad-cpp — отчёт по анализу порта

Сверка C++ реализации с оригинальной Python-библиотекой Soft-V/robocad-py (+ зависимость funcad)

Дата: 2026-06-24 · Проверено побайтно: сетевой протокол, упаковка структур, парсинг Titan/VMX, sim-каналы, shufflecad

Сводка

Серьёзность	Кол-во	Кратко
Критично	5	Не работает или падает: sim Algoritm, terminate при stop, сборка shufflecad под Linux, перепутанные каналы, detach+join
Серьёзно	7	Считает неверно: направления/PID, биты лимитов, power, inputs, deadlock лога, get_float, set_radar
Средне/риски	7	ODR-нарушение, не тот updater, LiDAR-заглушка, отсутствует CommonRobot, little-endian, include, recv

Что в порядке: Studica Titan TX/RX, Studica VMX RX (yaw/US/power/flex, calc_yaw_unlim), sim-канал Studica (структуры pack(1) точно совпадают с struct.pack/unpack), access_bit, камера в sim. Studica в симуляторе близка к рабочей после фикса №2.

Критичные проблемы (не работает / падает)

КРИТИЧНО

1. Симулятор Algoritm не реализован

Где: src/internal/algaritm_internal.cpp:62, :70

Проблема: Поток обновления sim-канала закомментирован, а update_loop() пустой (TODO). В Python RobocadConnection.__update отправляет скорости/сервы/шаги и читает энкодеры, ультразвук, аналоги, yaw/pitch/roll, лимиты, входы, статус шаговиков. В режиме симулятора Algoritm ничего не шлёт и не получает — все сенсоры остаются 0. Вдобавок структуры RobocadTxPacket/RobocadRxPacket скопированы из Studica.

```
// update_thread = std::thread(&RobocadConnectionAlgoritm::update_loop, this);
void update_loop() { /* TODO: */ }
```

Как должно быть: Реализовать update_loop с форматом Algoritm: TX 28 float (112 б), RX '<4i4f8H3f14B' (74 б), и раскомментировать запуск потока.

КРИТИЧНО**2. Падение при остановке (std::terminate) + use-after-free**

Где: src/internal/studica_internal.cpp:431, alaritm_internal.cpp:405

Проблема: stop() вызывает только connection->stop(), но НЕ останавливает рабочие потоки (RobocadConnection / TitanCOM / VMXSPI). В Python stop() останавливает их все. Деструктор делает delete connection раньше, чем останавливает поток, который в этот момент дергает connection->get_data() (use-after-free). У классов-обёрток нет деструктора, который join-ит std::thread □ уничтожение ещё joinable() потока вызывает std::terminate(). Воспроизводится при каждом завершении.

```
void StudicaInternal::stop() { connection->stop(); } // и всё
```

Как должно быть: В stop() останавливать все под-потоки ДО удаления connection; и/или вызывать stop()/join() в деструкторах обёрток.

КРИТИЧНО**3. Shufflecad не компилируется под Linux**

Где: src/shufflecad.cpp:228, :254, :264, :281, :311

Проблема: В non-Windows ветке макросов определены INVALID_SCT и SCT_ERROR, а в коде используются INVALID_SOCKET и SOCKET_ERROR — это имена из winsock, на Linux они не определены □ ошибка компиляции. Робот работает под Linux/RPi.

```
server_fd = INVALID_SOCKET; // не определено на Linux
if (bind(...) == SOCKET_ERROR) ... // не определено на Linux
```

Как должно быть: Использовать INVALID_SCT / SCT_ERROR (как объявлено), либо определить INVALID_SOCKET/SOCKET_ERROR для non-Windows.

КРИТИЧНО**4. Shufflecad: перепутаны обработчики каналов**

Где: src/shufflecad.cpp:335-339

Проблема: Каналы chart/outcad/rpi подключены к on_in_vars, а joystick — к on_camera_vars. В итоге графики, лог (outcad) и телеметрия RPi (температура/CPU/power) никогда не отправляются, а джойстик не парсится. Функции on_chart_vars/on_outcad_vars/on_rpi_vars/on_joy_vars объявлены, но не подключены.

```
chart_variables_channel = new TalkPort(63255, on_in_vars, ...); // -> on_chart_vars
outcad_variables_channel = new TalkPort(63257, on_in_vars, ...); // -> on_outcad_vars
rpi_variables_channel = new TalkPort(63256, on_in_vars, ...); // -> on_rpi_vars
joy_variables_channel = new ListenPort(63259, on_camera_vars); // -> on_joy_vars
```

Как должно быть: Подключить корректные обработчики к соответствующим каналам.

КРИТИЧНО**5. ConnectionReal: detach() затем join()**

Где: src/internal/common/connection_real.cpp:39 vs :61

Проблема: В конструкторе поток robot_info_thread делается detach(), а в stop() вызывается join(). После detach() поток не joinable(), и join() бросит std::system_error. (Путь только для реального робота.)

```
robot_info_thread->detach(); // в конструкторе
...
robot_info_thread->join(); // в stop() -> исключение
```

Как должно быть: Не вызывать detach(), если планируется join() в stop().

Серьёзные ошибки логики (компилируется, но считает неверно)

СЕРЬЁЗНО**6. Algoritm Titan: направления шаговиков и PID всегда «вкл»**

Где: src/internal/algoritm_internal.cpp:186

Проблема: Условие `bool >= 0` всегда истинно, поэтому биты направлений шаговиков и флага PID выставляются безусловно, независимо от реальных значений.

```
if (robot_internal->step_motor_1_direction >= 0) dir_byte |= (1 << 5);
if (robot_internal->step_motor_2_direction >= 0) dir_byte |= (1 << 4);
if (robot_internal->use_pid >= 0) dir_byte |= (1 << 3);
```

Как должно быть: Использовать просто `if (step_motor_1_direction)` и т.д.

СЕРЬЁЗНО**7. Algoritm Titan: лимиты читаются с зеркальных бит**

Где: src/internal/algoritm_internal.cpp:152

Проблема: Индексы бит развёрнуты. Python: `limit_l_0=bit7, limit_h_0=bit6, ... limit_h_3=bit0`. C++: `limit_l_0=bit0, ... limit_h_3=bit7`. Все лимиты Titan на Algoritm читаются неверно.

```
// C++ (неверно):          // Python (эталон):
limit_l_0 = access_bit(data[17], 0); // access_bit(data[17], 7)
limit_h_3 = access_bit(data[17], 7); // access_bit(data[17], 0)
```

Как должно быть: Развернуть индексы бит: `limit_l_0 = access_bit(data[17], 7)`, и т.д.

СЕРЬЁЗНО**8. Algoritm VMX: power читается из неверных байт**

Где: src/internal/algoritm_internal.cpp:330

Проблема: В ветке `data[0]==3` power берётся из `data[8]/data[7]` (скопировано из Studica), а в Python — из `data[9]/data[8]`.

```
double power = ((data[8] << 8) | data[7]) / 100.0; // должно быть data[9]/data[8]
```

Как должно быть: `power = ((data[9] << 8) | data[8]) / 100.0;`

СЕРЬЁЗНО**9. Algoritm VMX: не читаются входы (inputs)**

Где: src/internal/algoritm_internal.cpp:300 (ветка `data[0]==2`)

Проблема: В Python после ультразвуков читаются 4 входа из бит `data[11]`. В C++ этого блока нет вообще — inputs на реальном роботе никогда не обновляются.

```
// Python:
self.inputs[0] = Funcad.access_bit(data[11], 0) == 1 # ... [1..3]
// В C++ отсутствует
```

Как должно быть: Добавить чтение `inputs[0..3] = access_bit(data[11], 0..3)`.

СЕРЬЁЗНО**10. write_log: deadlock при незакрытом лог-файле**

Где: src/internal/common/robot.cpp:47

Проблема: При незакрытом файле функция выходит, не отпустив мьютекс `log_mutex` все последующие `write_log` зависнут навсегда.

```
log_mutex.lock();
if (!log_file.is_open()) return; // возврат с захваченным мьютексом!
...
log_mutex.unlock();
```

Как должно быть: Использовать `std::lock_guard` вместо ручных `lock()/unlock()`.

СЕРЬЁЗНО**11. ShuffleVariable::get_float — краш на пустом значении**

Где: include/shufflecad.hpp:110

Проблема: std::stof на пустой/нечисловой строке бросает std::invalid_argument (неперехвачено) ☐ краш. Python ловит исключения и возвращает 0, а также заменяет ',' на '.'.

```
float get_float() { ...; return std::stof(this->value); }
```

Как должно быть: Обернуть в try/catch, вернуть 0 при ошибке, заменить запятую на точку.

СЕРЬЁЗНО**12. ShuffleVariable::set_radar — неверный формат строки**

Где: include/shufflecad.hpp:93

Проблема: Между индексом и значением не вставляется '+'. Python даёт "0+v0+1+v1+...", C++ — "0v0+1v1+...". Радар-переменная придёт в shufflecad битой.

```
val += std::to_string(i);
val += std::to_string(values[i]); // нет '+' между индексом и значением
```

Как должно быть: Собирать как чередование index/value через '+', как в Python.

Средние проблемы и риски

ODR-нарушение классов

connection.hpp / shufflecad.cpp

Классы ListenPort, TalkPort, SocketInit определены по-разному в двух местах. Деструктор ~ListenPort() имеет одинаковый символ, но разные тела ☐ формально UB, линкер может выбрать «не тот». Решение: анонимный namespace или переименование в shufflecad.

Algaritm использует не тот updater

algaritm_internal.cpp:369

Создаётся new RpiUpdater, а в Python — RepkaUpdater. На практике обе C++-реализации идентичны (читают /sys/.../thermal_zone0/temp), эффект нулевой, но это расхождение.

LiDAR — заглушка

src/internal/common/lidar.cpp

Все методы пустые (// TODO). На реальном Algaritm get_lidar()/lidar_data всегда вернёт пусто.

CommonRobot не портирован

(нет common.cpp/.hpp)

В Python есть common.py (CommonRobot, sim-only, 8 моторов). В C++ отсутствует целиком.

Зависимость от little-endian

connection.hpp:124 и др.

Длина шлётся как &uint32; напрямую, float-структуры читаются reinterpret_cast. Python жёстко фиксирует '<'. На x86/ARM (LE) совпадает, но переносимость потеряна.

robot.hpp: нет #include

include/internal/common/robot.hpp:11

Используется std::atomic без явного include — может не собраться на части стандартных библиотек.

TalkPort: ответ читается одним recv(8)

connection.hpp:223

Вместо двух recv(4), как в Python. При коротком чтении возможна рассинхронизация; на localhost обычно ок.

Рекомендуемый порядок исправления: сначала №1–5 (критичные), затем №6–9 (логика Algaritm), затем №10–12 (shufflecad/лог), после — средние. Для реализации sim-канала Algaritm (№1) нужно описание сетевого протокола симулятора robocad, чтобы повторить формат как в Python.